

labsheet-1

```
In [1]: import cv2
import numpy as np
import matplotlib.pyplot as plt

print("=" * 60)
print("FUNDAMENTAL IMAGE PROCESSING OPERATIONS")
print("=" * 60)

# -----
# 1. READ IMAGE
# -----
image_path = input("Enter image path: ")

image = cv2.imread(image_path)

if image is None:
    print("Error: Image not found!")
    print("Please check the image path.")
    exit()

print("Image loaded successfully!")

# -----
# 2. DISPLAY ORIGINAL IMAGE
# -----
rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(7, 5))
plt.imshow(rgb_image)
plt.title("Original Image")
plt.axis("off")
plt.show()

# -----
# 3. IMAGE PROPERTIES
# -----
height, width, channels = image.shape

print("\nImage Properties")
print("-" * 30)
print("Width      :", width)
print("Height     :", height)
print("Channels    :", channels)
print("Data Type   :", image.dtype)
print("Resolution:", width, "x", height)

# -----
# 4. SAVE IMAGE
# -----
cv2.imwrite("output.jpg", image)
cv2.imwrite("output.png", image)

print("\nImage saved as output.jpg and output.png")

# -----
# 5. GRAYSCALE CONVERSION
```

```
# -----
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(7, 5))
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.show()

# -----
# 6. HSV COLOR SPACE
# -----

hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# Convert HSV for display
hsv_display = cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB)

plt.figure(figsize=(7, 5))
plt.imshow(hsv_display)
plt.title("HSV Image")
plt.axis("off")
plt.show()

# -----
# 7. LAB COLOR SPACE
# -----

lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)

lab_display = cv2.cvtColor(lab, cv2.COLOR_LAB2RGB)

plt.figure(figsize=(7, 5))
plt.imshow(lab_display)
plt.title("LAB Image")
plt.axis("off")
plt.show()

# -----
# 8. RESIZE IMAGE
# -----

resized = cv2.resize(image, (400, 300))

resized_rgb = cv2.cvtColor(resized, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(7, 5))
plt.imshow(resized_rgb)
plt.title("Resized Image")
plt.axis("off")
plt.show()

# -----
# 9. ROTATE IMAGE
# -----

height, width = image.shape[:2]

center = (width // 2, height // 2)

rotation_matrix = cv2.getRotationMatrix2D(
    center,
    45,
    1.0
```

```
)

rotated = cv2.warpAffine(
    image,
    rotation_matrix,
    (width, height)
)

rotated_rgb = cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(7, 5))
plt.imshow(rotated_rgb)
plt.title("Rotated Image - 45 Degrees")
plt.axis("off")
plt.show()

# -----
# 10. HORIZONTAL FLIP
# -----
horizontal_flip = cv2.flip(image, 1)

horizontal_flip_rgb = cv2.cvtColor(
    horizontal_flip,
    cv2.COLOR_BGR2RGB
)

plt.figure(figsize=(7, 5))
plt.imshow(horizontal_flip_rgb)
plt.title("Horizontal Flip")
plt.axis("off")
plt.show()

# -----
# 11. VERTICAL FLIP
# -----
vertical_flip = cv2.flip(image, 0)

vertical_flip_rgb = cv2.cvtColor(
    vertical_flip,
    cv2.COLOR_BGR2RGB
)

plt.figure(figsize=(7, 5))
plt.imshow(vertical_flip_rgb)
plt.title("Vertical Flip")
plt.axis("off")
plt.show()

# -----
# 12. NEGATIVE IMAGE
# -----
negative = 255 - image

negative_rgb = cv2.cvtColor(
    negative,
    cv2.COLOR_BGR2RGB
)

plt.figure(figsize=(7, 5))
plt.imshow(negative_rgb)
```

```
plt.title("Negative Image")
plt.axis("off")
plt.show()

# -----
# 13. REGION OF INTEREST (ROI)
# -----
height, width = image.shape[:2]

x1 = width // 4
x2 = 3 * width // 4

y1 = height // 4
y2 = 3 * height // 4

roi = image[y1:y2, x1:x2]

roi_rgb = cv2.cvtColor(
    roi,
    cv2.COLOR_BGR2RGB
)

plt.figure(figsize=(7, 5))
plt.imshow(roi_rgb)
plt.title("Region of Interest (ROI)")
plt.axis("off")
plt.show()

# -----
# 14. DISPLAY RESULTS TOGETHER
# -----
plt.figure(figsize=(15, 10))

plt.subplot(2, 4, 1)
plt.imshow(rgb_image)
plt.title("Original")
plt.axis("off")

plt.subplot(2, 4, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(2, 4, 3)
plt.imshow(resized_rgb)
plt.title("Resized")
plt.axis("off")

plt.subplot(2, 4, 4)
plt.imshow(rotated_rgb)
plt.title("Rotated")
plt.axis("off")

plt.subplot(2, 4, 5)
plt.imshow(horizontal_flip_rgb)
plt.title("Horizontal Flip")
plt.axis("off")

plt.subplot(2, 4, 6)
plt.imshow(vertical_flip_rgb)
```

```
plt.title("Vertical Flip")
plt.axis("off")

plt.subplot(2, 4, 7)
plt.imshow(negative_rgb)
plt.title("Negative")
plt.axis("off")

plt.subplot(2, 4, 8)
plt.imshow(roi_rgb)
plt.title("ROI")
plt.axis("off")

plt.tight_layout()
plt.show()

print("\n" + "=" * 60)
print("EXPERIMENT COMPLETED SUCCESSFULLY")
print("=" * 60)
```

FUNDAMENTAL IMAGE PROCESSING OPERATIONS

Image loaded successfully!

Original Image



Image Properties

Width : 1536
Height : 1024
Channels : 3
Data Type : uint8
Resolution: 1536 x 1024

Image saved as output.jpg and output.png

Grayscale Image

Original Image



(Grayscale)

Low-Pass Filters (Smoothing & Noise Reduction)

Gaussian Blur



(Reduces noise, preserves structure)

Median Filter



(Removes salt-and-pepper noise, keeps edges)

Average Filter



(Simple smoothing, can blur edges)

High-Pass Filters (Edge Enhancement & Feature Detection)

Laplacian Filter



(Enhances edges and fine details)

Sobel X (Vertical Edges)



(Detects vertical edges)

Sobel Y (Horizontal Edges)



(Detects horizontal edges)

Edge Enhanced (Laplacian)



(Sharper details and edges)

Salt-and-Pepper Noise Removal (Median Filter)

Original



(Clean image)

Noisy Image



(Salt-and-pepper noise)

Median Filtered



(Noise removed, edges preserved)

Filter Comparison					
Filter	Type	Noise Reduction	Edge Preservation	Edge Enhancement	Main Application
Gaussian	Low-pass	Good	Good	Low	Smoothing
Median	Low-pass	Excellent (impulse noise)	Very Good	Low	Salt and pepper noise removal
Average	Low-pass	Good	Poor	Low	Simple smoothing
Laplacian	High-pass	Poor	Moderate	Excellent	Sharpening, edge enhancement
Sobel X	High-pass	Poor	Good	Excellent	Vertical edges
Sobel Y	High-pass	Poor	Good	Excellent	Horizontal edges

Key Observations

1. Gaussian filtering provides smooth results and is effective for reducing random noise.
2. Median filtering is highly effective against salt and pepper noise and preserves edges better than average filtering.
3. Average filtering smooths the image but tends to blur object boundaries.
4. Laplacian filtering emphasizes edges and fine details but can also amplify noise.
5. Sobel X detects vertical edges, while Sobel Y detects horizontal edges.
6. Low-pass filters are suitable for noise reduction and smoothing, whereas high-pass filters are useful for edge detection and sharpening.

HSV Image

Original Image



(Grayscale)

Low-Pass Filters (Smoothing & Noise Reduction)

Gaussian Blur



(Reduces noise, preserves structure)

Median Filter



(Removes salt-and-pepper noise, keeps edges)

Average Filter



(Simple smoothing, can blur edges)

High-Pass Filters (Edge Enhancement & Feature Detection)

Laplacian Filter



(Enhances edges and fine details)

Sobel X (Vertical Edges)



(Detects vertical edges)

Sobel Y (Horizontal Edges)



(Detects horizontal edges)

Edge Enhanced (Laplacian)



(Sharper details and edges)

Salt-and-Pepper Noise Removal (Median Filter)

Original



(Clean image)

Noisy Image



(Salt-and-pepper noise)

Median Filtered



(Noise removed, edges preserved)

Filter Comparison					
Filter	Type	Noise Reduction	Edge Preservation	Edge Enhancement	Main Application
Gaussian	Low-pass	Good	Good	Low	Smoothing
Median	Low-pass	Excellent (impulse noise)	Very Good	Low	Salt and pepper noise removal
Average	Low-pass	Good	Poor	Low	Simple smoothing
Laplacian	High-pass	Poor	Moderate	Excellent	Sharpening, edge enhancement
Sobel X	High-pass	Poor	Good	Excellent	Vertical edges
Sobel Y	High-pass	Poor	Good	Excellent	Horizontal edges

Key Observations

1. Gaussian filtering provides smooth results and is effective for reducing random noise.
2. Median filtering is highly effective against salt and pepper noise and preserves edges better than average filtering.
3. Average filtering smooths the image but tends to blur object boundaries.
4. Laplacian filtering emphasizes edges and fine details but can also amplify noise.
5. Sobel X detects vertical edges, while Sobel Y detects horizontal edges.
6. Low-pass filters are suitable for noise reduction and smoothing, whereas high-pass filters are useful for edge detection and sharpening.

127.0.0.1:5500/labsheet-1.html

6/21

LAB Image

Original Image



(Grayscale)

Low-Pass Filters (Smoothing & Noise Reduction)

Gaussian Blur



(Reduces noise, preserves structure)

Median Filter



(Removes salt-and-pepper noise, keeps edges)

Average Filter



(Simple smoothing, can blur edges)

High-Pass Filters (Edge Enhancement & Feature Detection)

Laplacian Filter



(Enhances edges and fine details)

Sobel X (Vertical Edges)



(Detects vertical edges)

Sobel Y (Horizontal Edges)



(Detects horizontal edges)

Edge Enhanced (Laplacian)



(Sharper details and edges)

Salt-and-Pepper Noise Removal (Median Filter)

Original



(Clean image)

Noisy Image



(Salt-and-pepper noise)

Median Filtered



(Noise removed, edges preserved)

Filter Comparison

Filter	Type	Noise Reduction	Edge Preservation	Edge Enhancement	Main Application
Gaussian	Low-pass	Good	Good	Low	Smoothing
Median	Low-pass	Excellent (Impulse noise)	Very Good	Low	Salt-and-pepper noise removal
Average	Low-pass	Good	Poor	Low	Simple smoothing
Laplacian	High-pass	Poor	Moderate	Excellent	Sharpening, edge enhancement
Sobel X	High-pass	Poor	Good	Excellent	Vertical edges
Sobel Y	High-pass	Poor	Good	Excellent	Horizontal edges

Key Observations

- Gaussian filtering provides smooth results and is effective for reducing random noise.
- Median filtering is highly effective against salt-and-pepper noise and preserves edges better than average filtering.
- Average filtering smooths the image but tends to blur object boundaries.
- Laplacian filtering emphasizes edges and fine details but can also amplify noise.
- Sobel X detects vertical edges, while Sobel Y detects horizontal edges.
- Low-pass filters are suitable for noise reduction and smoothing, whereas high-pass filters are useful for edge detection and sharpening.

Resized Image

Original Image



(Grayscale)

Low-Pass Filters (Smoothing & Noise Reduction)

Gaussian Blur



(Reduces noise, preserves structure)

Median Filter



(Removes salt-and-pepper noise, keeps edges)

Average Filter



(Simple smoothing, can blur edges)

High-Pass Filters (Edge Enhancement & Feature Detection)

Laplacian Filter



(Enhances edges and fine details)

Sobel X (Vertical Edges)



(Detects vertical edges)

Sobel Y (Horizontal Edges)



(Detects horizontal edges)

Edge Enhanced (Laplacian)



(Sharper details and edges)

Salt-and-Pepper Noise Removal (Median Filter)

Original



(Clean image)

Noisy Image



(Salt-and-pepper noise)

Median Filtered



(Noise removed, edges preserved)

Filter Comparison

Filter	Type	Noise Reduction	Edge Preservation	Edge Enhancement	Main Application
Gaussian	Low-pass	Good	Good	Low	Smoothing
Median	Low-pass	Excellent (Impulse noise)	Very Good	Low	Salt-and-pepper noise removal
Average	Low-pass	Good	Poor	Low	Simple smoothing
Laplacian	High-pass	Poor	Moderate	Excellent	Sharpening, edge enhancement
Sobel X	High-pass	Poor	Good	Excellent	Vertical edges
Sobel Y	High-pass	Poor	Good	Excellent	Horizontal edges

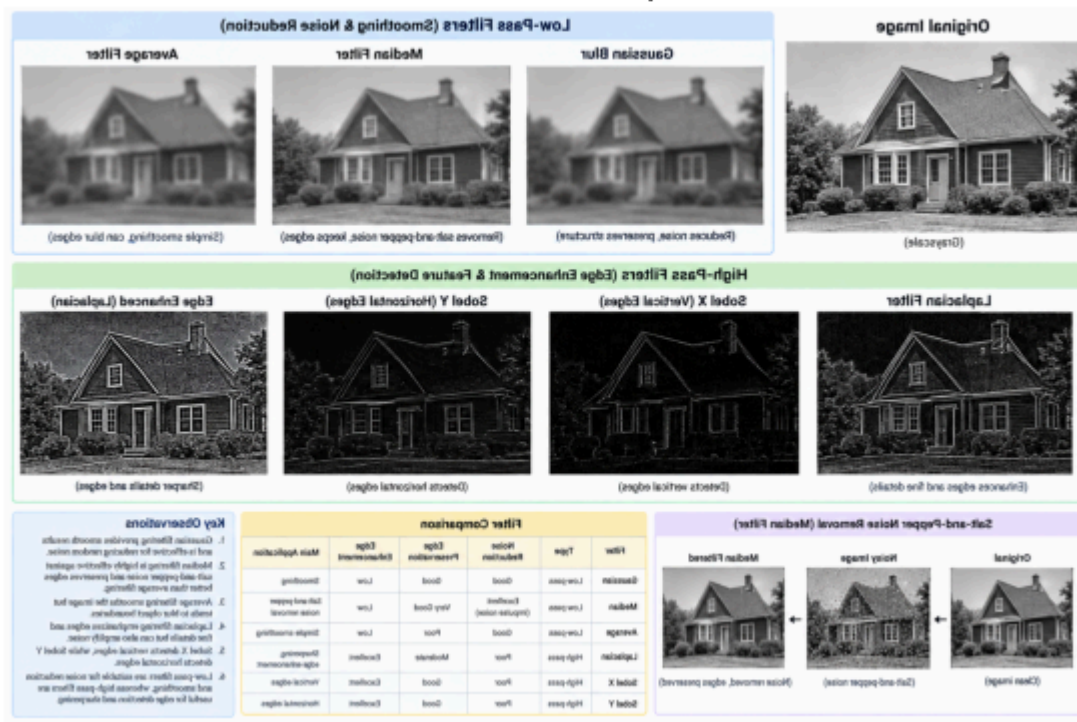
Key Observations

- Gaussian filtering provides smooth results and is effective for reducing random noise.
- Median filtering is highly effective against salt-and-pepper noise and preserves edges better than average filtering.
- Average filtering smooths the image but tends to blur object boundaries.
- Laplacian filtering emphasizes edges and fine details but can also amplify noise.
- Sobel X detects vertical edges, while Sobel Y detects horizontal edges.
- Low-pass filters are suitable for noise reduction and smoothing, whereas high-pass filters are useful for edge detection and sharpening.

Rotated Image - 45 Degrees



Horizontal Flip



Vertical Flip

(Original image) (Flipped image) (Flipped image)

Original image (upside down) Flipped image (upside up) Flipped image (upside down)

Filter	Type	Noise Reduction	Edge Preservation	Edge Enhancement	Main Application
Gaussian	Low-pass	Good	Good	Low	Smoothing
Median	Low-pass	Excellent (impulse noise)	Very Good	Low	Salt and pepper noise removal
Average	Low-pass	Good	Poor	Low	Simple smoothing
Laplacian	High-pass	Poor	Moderate	Excellent	Sharpening, edge enhancement
Sobel X	High-pass	Poor	Good	Excellent	Vertical edge detection
Sobel Y	High-pass	Poor	Good	Excellent	Horizontal edge detection

Key Observations

1. Gaussian filtering provides smooth results and is effective for reducing random noise.
2. Median filtering is highly effective against salt and pepper noise and preserves edges better than average filtering.
3. Average filtering smooths the image but tends to blur object boundaries.
4. Laplacian filtering emphasizes edges and fine details but can also amplify noise.
5. Sobel X detects vertical edges, while Sobel Y detects horizontal edges.
6. Low-pass filters are suitable for noise reduction and smoothing, whereas high-pass filters are useful for edge detection and sharpening.

Laplacian Filter

Sobel X (Vertical Edges)

Sobel Y (Horizontal Edges)

Edge Enhanced (Laplacian)

High-Pass Filters (Edge Enhancement & Feature Detection)

Original image

Gaussian Blur

Median Filter

Average Filter

Low-Pass Filters (Smoothing & Noise Reduction)

Negative Image

Original Image

(Grayscale)

Low-Pass Filters (Smoothing & Noise Reduction)

Gaussian Blur

(Reduces noise, preserves structure)

Median Filter

(Removes salt and pepper noise, keeps edges)

Average Filter

(Simple smoothing, can blur edges)

High-Pass Filters (Edge Enhancement & Feature Detection)

Laplacian Filter

(Enhances edges and fine details)

Sobel X (Vertical Edges)

(Detects vertical edges)

Sobel Y (Horizontal Edges)

(Detects horizontal edges)

Edge Enhanced (Laplacian)

(Sharper details and edges)

Salt-and-Pepper Noise Removal (Median Filter)

(Clean image)

(Salt and pepper noise)

(Noise removed, edges preserved)

Filter	Type	Noise Reduction	Edge Preservation	Edge Enhancement	Main Application
Gaussian	Low-pass	Good	Good	Low	Smoothing
Median	Low-pass	Excellent (impulse noise)	Very Good	Low	Salt and pepper noise removal
Average	Low-pass	Good	Poor	Low	Simple smoothing
Laplacian	High-pass	Poor	Moderate	Excellent	Sharpening, edge enhancement
Sobel X	High-pass	Poor	Good	Excellent	Vertical edge detection
Sobel Y	High-pass	Poor	Good	Excellent	Horizontal edge detection

Key Observations

1. Gaussian filtering provides smooth results and is effective for reducing random noise.
2. Median filtering is highly effective against salt and pepper noise and preserves edges better than average filtering.
3. Average filtering smooths the image but tends to blur object boundaries.
4. Laplacian filtering emphasizes edges and fine details but can also amplify noise.
5. Sobel X detects vertical edges, while Sobel Y detects horizontal edges.
6. Low-pass filters are suitable for noise reduction and smoothing, whereas high-pass filters are useful for edge detection and sharpening.

Region of Interest (ROI)



High-Pass Filters (Edge Enhancement & Feature Detection)

Sobel X (Vertical Edges)



(Detects vertical edges)

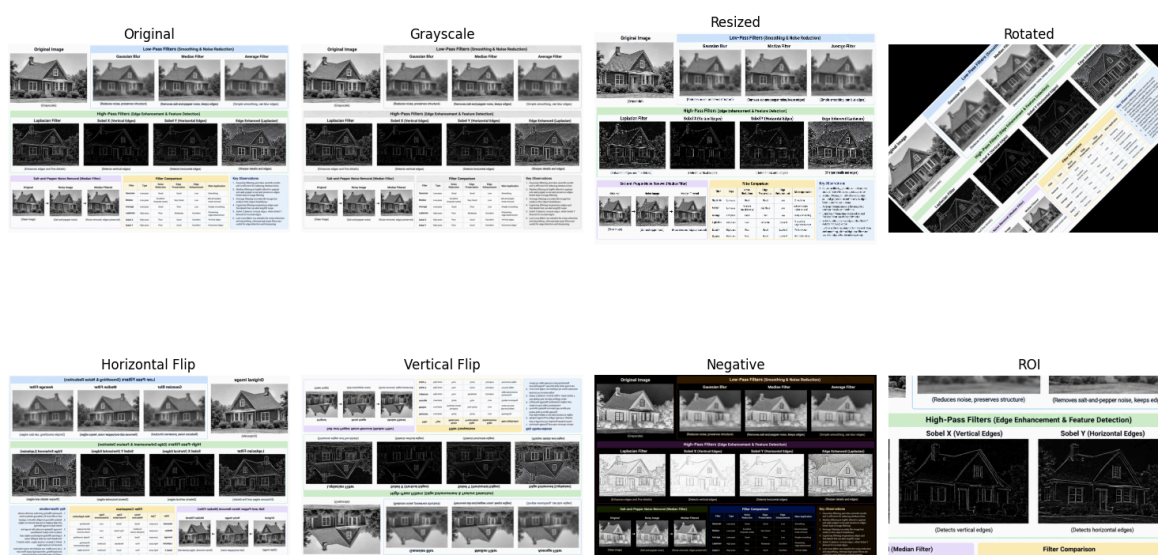
Sobel Y (Horizontal Edges)



(Detects horizontal edges)

(Median Filter)

Filter Comparison



=====

EXPERIMENT COMPLETED SUCCESSFULLY

=====

PART B – QUESTIONS WITH SOLUTIONS

Q1. What is a digital image? Differentiate between grayscale and color images.

Answer:

A **digital image** is a two-dimensional representation of a visual scene that is stored and processed in digital form. It is composed of small elements called **pixels (picture elements)**. Each pixel contains information about the brightness or color at a particular location in the image.

A digital image can be represented mathematically as a function:

$f(x, y)$

where x and y represent the spatial coordinates of a pixel and $f(x, y)$ represents its intensity or color value.

Grayscale Image

A grayscale image contains only intensity information. Each pixel represents a shade of gray.

For an 8-bit grayscale image:

- 0 represents black.
- 255 represents white.
- Values between 0 and 255 represent different shades of gray.

A grayscale image generally has **one channel**.

Color Image

A color image contains color information. A commonly used representation is RGB, which has three channels:

- Red
- Green
- Blue

Different combinations of these channels produce different colors.

Difference between Grayscale and Color Images

Grayscale Image	Color Image
Contains one channel	Usually contains three channels
Represents intensity/brightness	Represents color information
Values generally range from 0–255	Each color channel generally ranges from 0–255
Requires less storage	Requires more storage
Easier and faster to process	More computationally expensive
Used when color is not important	Used when color information is required

Q2. Explain the difference between RGB, BGR, HSV, and LAB color spaces.

Answer:

A **color space** is a method of representing the colors of an image using numerical values.

RGB

RGB stands for:

- **R – Red**
- **G – Green**
- **B – Blue**

RGB is commonly used for displaying images on computer screens and other digital devices.

BGR

BGR stands for:

- **B – Blue**
- **G – Green**
- **R – Red**

BGR contains the same three color components as RGB, but the order of the channels is different. **OpenCV normally reads color images in BGR format.**

HSV

HSV stands for:

- **H – Hue**
- **S – Saturation**
- **V – Value**

Hue represents the type of color, saturation represents the purity/intensity of the color, and value represents brightness.

HSV is particularly useful for **color detection and segmentation**.

LAB

LAB consists of:

- **L – Lightness**
- **A – Green to Red**
- **B – Blue to Yellow**

LAB separates lightness from color information and is useful for **color analysis**.

Comparison of Color Spaces

Color Space	Components	Main Use
RGB	Red, Green, Blue	Image display
BGR	Blue, Green, Red	OpenCV image processing
HSV	Hue, Saturation, Value	Color detection/segmentation
LAB	Lightness, A, B	Color analysis

OpenCV Example

```
import cv2

image = cv2.imread("image.jpg")

rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)
```

Q3. What is the purpose of converting an image to grayscale before further processing?

Answer:

Converting a color image to grayscale means converting an image with three color channels into a single intensity channel.

The main purpose is to simplify image processing.

Advantages of grayscale conversion:

1. **Reduces computational complexity** because only one channel needs to be processed.
2. **Reduces memory requirements** compared with a three-channel color image.
3. **Simplifies mathematical operations.**
4. Removes color information when it is not required.
5. Makes many image-processing algorithms easier to apply.
6. Improves processing speed.
7. It is commonly used before edge detection, thresholding, contour detection, and other operations.

OpenCV Code

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

For an 8-bit grayscale image, pixel intensity values normally range from:

0 → Black

255 → White

Therefore, grayscale conversion is an important preprocessing step in many computer vision applications.

Q4. Explain the concept of image complement (negative image) and mention its practical applications.

Answer:

An **image complement**, also called an **image negative**, is obtained by reversing the intensity values of an image.

For an 8-bit image, the negative value can be calculated as:

$$\text{Negative} = 255 - \text{Original Pixel Value}$$

For example:

$$\text{Original pixel} = 50$$

$$\begin{aligned}\text{Negative} &= 255 - 50 \\ &= 205\end{aligned}$$

Similarly:

$$\text{Original pixel} = 200$$

$$\begin{aligned}\text{Negative} &= 255 - 200 \\ &= 55\end{aligned}$$

Therefore, dark regions become bright and bright regions become dark.

OpenCV/Python Code

```
negative = 255 - image
```

Applications of Image Complement

Image negatives can be useful in:

1. Medical image visualization.
 2. X-ray image processing.
 3. Image enhancement.
 4. Highlighting details in dark regions.
 5. Photograph processing.
 6. Scientific image analysis.
 7. Visualization of specific image features.
-

Q5. Differentiate between image resizing, cropping, and scaling.

Answer:

Image resizing, cropping, and scaling are geometric operations used to modify an image.

1. Image Resizing

Resizing changes the dimensions of an image to a specified width and height.

For example, an image can be changed from:

800 × 600

to:

400 × 300

Python code:

```
resized = cv2.resize(image, (400, 300))
```

2. Image Cropping

Cropping means selecting a particular portion of an image and removing the remaining portion.

Example:

```
cropped = image[100:300, 150:400]
```

3. Image Scaling

Scaling changes the size of an image according to a scale factor.

For example:

```
scaled = cv2.resize(  
    image,  
    None,  
    fx=0.5,  
    fy=0.5  
)
```

Here, the width and height are reduced to 50%.

Difference

Operation	Description
Resizing	Changes image dimensions
Cropping	Selects a particular region of the image
Scaling	Changes image size according to a scale factor

Q6. What is a Region of Interest (ROI), and why is it important in computer vision?

Answer:

A **Region of Interest (ROI)** is a specific portion of an image that contains the important area that needs to be processed or analyzed.

Instead of processing the entire image, computer vision algorithms can focus only on the required region.

Example

```
roi = image[100:300, 150:400]
```

Here, a rectangular section of the image is selected as the ROI.

Importance of ROI

ROI is important because:

1. It reduces the amount of data that needs to be processed.
2. It reduces computational cost.
3. It improves processing efficiency.
4. It focuses the algorithm on the important part of the image.
5. It is useful in object detection.
6. It is useful in face detection.
7. It is used in image segmentation.
8. It can improve the accuracy of image analysis.

Example Application

In face recognition, the complete image does not always need to be analyzed. Once the face region is identified, further processing can be performed only on that ROI.

Q7. Why is OpenCV preferred over conventional image processing libraries for computer vision applications?

Answer:

OpenCV (Open Source Computer Vision Library) is a widely used library for image processing and computer vision.

It provides a large collection of functions for processing images and videos.

Advantages of OpenCV

1. It is open-source and freely available.
2. It supports Python and several other programming languages.
3. It provides efficient image-processing functions.
4. It supports image reading and writing.
5. It supports image resizing, rotation, cropping, and flipping.
6. It provides color-space conversion functions.
7. It supports filtering and image enhancement.
8. It provides feature detection and extraction techniques.
9. It supports object detection and recognition.
10. It can be used for real-time computer vision applications.
11. It supports image and video processing.
12. It is widely used in academic and industrial applications.

Example

```
import cv2
```

```
image = cv2.imread("image.jpg")  
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

Because of its extensive functionality and efficient implementation, OpenCV is commonly preferred for computer vision applications.

Q8. Explain how image resolution and pixel intensity influence image quality.

Answer:

Two important factors that influence digital image quality are **image resolution** and **pixel intensity**.

Image Resolution

Image resolution refers to the number of pixels present in an image.

For example:

1920×1080

means that the image contains 1920 pixels horizontally and 1080 pixels vertically.

A higher-resolution image can generally represent more visual details.

Low Resolution

- Contains fewer pixels.
- May lose details.
- Can appear pixelated when enlarged.

High Resolution

- Contains more pixels.
- Can represent more details.
- Usually provides better detail when viewed at an appropriate size.

Pixel Intensity

Pixel intensity represents the brightness of an individual pixel.

For an 8-bit grayscale image:

0 → Black

255 → White

Values between 0 and 255 represent different shades of gray.

Influence on Image Quality

- Resolution affects the amount of spatial detail.
- Intensity affects brightness.
- The range of intensity values affects contrast.
- Poor intensity distribution can make an image too dark, too bright, or low in contrast.

Therefore, both resolution and pixel intensity are important for image quality.

Q9. Mention five real-world applications where basic image preprocessing is an essential step.

Answer:

Basic image preprocessing is an important step in many computer vision applications.

1. Medical Imaging

Medical images such as X-rays, CT scans, and MRI images can be processed to improve their quality before analysis.

Examples of preprocessing include:

- Grayscale conversion

- Noise removal
- Contrast enhancement
- Resizing

2. Face Recognition

Images are often resized, normalized, converted to grayscale, or enhanced before face recognition algorithms process them.

3. Autonomous Vehicles

Cameras in autonomous vehicles capture images that must be processed before detecting:

- Vehicles
- Pedestrians
- Roads
- Traffic signs
- Lane markings

4. Industrial Inspection

Image preprocessing is used to identify defects in manufactured products.

Examples include:

- Cracks
- Scratches
- Missing components
- Surface defects

5. Object Detection

Preprocessing prepares images before object detection algorithms analyze them.

Typical operations include:

- Resizing
- Normalization
- Noise reduction
- Color-space conversion

Other Applications

Basic image preprocessing is also used in:

- OCR and document recognition
- Satellite image analysis
- Robotics

- Security systems
 - Image classification
 - Traffic monitoring
-

Q10. How do image preprocessing techniques improve the performance of feature extraction and deep learning models?

Answer:

Image preprocessing improves the quality and consistency of image data before it is given to feature extraction algorithms or deep learning models.

Important preprocessing techniques include:

1. Resizing

Images may have different dimensions. Resizing converts them to a common size.

```
image = cv2.resize(image, (224, 224))
```

This is especially useful when a model requires a fixed input size.

2. Grayscale Conversion

If color information is not required, converting an image to grayscale reduces the number of channels.

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

3. Normalization

Pixel values can be scaled to a standard range.

For example:

```
normalized = image / 255.0
```

This converts pixel values from approximately 0-255 to 0-1 .

4. Noise Reduction

Noise can interfere with feature extraction. Filtering techniques can be used to reduce unwanted variations.

5. Contrast Enhancement

Contrast enhancement can make important structures and features more visible.

6. Color-Space Conversion

An appropriate color space such as HSV or LAB can be used when color information is important.

Benefits

Image preprocessing can:

- Improve image quality.
- Remove unwanted information.
- Standardize input images.
- Reduce computational complexity.
- Improve the quality of extracted features.
- Make model training more stable.
- Help machine learning and deep learning models learn useful patterns.

Therefore, preprocessing is an important step before feature extraction and deep learning.

CONCLUSION

Fundamental image processing operations provide the foundation for computer vision. In this experiment, concepts such as digital images, grayscale and color images, RGB/BGR/HSV/LAB color spaces, image complement, resizing, cropping, scaling, ROI, image resolution, and pixel intensity were studied.

These techniques are commonly used as preprocessing steps before applying advanced computer vision methods such as **feature extraction, object detection, image classification, and deep learning**.